LR-Agent / Agent 后端入门

第06章
MCP子代理与上下文记忆

工具发现、只读委派与信息注入，理解扩展能力的真实边界

读者起点与本章边界

前置：已完成第 1、2 章基础实验。只围绕后端输入、决策、执行与验证学习；遇到新名词先看定义和小例子。真实截图、源码阅读图与概念示意分别注明。

学完本章应该能够

- 区分 MCP 协议、服务与工具
- 解释发现缓存、去重和记忆暴露策略
- 验证只读子代理双白名单
- 区分 Context、摘要、业务记忆与图 Store

源码核对：2026-10-17，工作区基线 d5dd009；本轮包含已验证的日志断管道修复。正式环境 LangChain 1.4.2 / LangGraph 1.2.12。

本章目录与实验准备

6.1 先问缺什么能力，再决定如何扩展 . . . 3
6.2 MCP：协议、服务和工具不能混为一谈 . . . 4
6.3 从配置到工具清单：四步读懂装配 . . . 5
6.4 缓存与降级：别让扩展拖住每次请求 . . . 6
6.5 去重与能力：当前保证到哪一步 . . . 7
6.6 完整实验一：记忆开关改变哪些工具 . . . 8
6.7 只读子代理：把一次查阅封装成工具 . . . 9
6.8 只读限制：工具清单与执行表一起过滤 . . . 10
6.9 完整实验二：亲眼看双白名单生效 . . . 11
6.10 custom 通道：进度流与父消息分开走 . . . 12
6.11 上下文、摘要、记忆、Store 逐个分清 . . . 13
6.12 指令、Skills 和记忆索引如何进提示词 . . . 14
6.13 完整实验三：验证提示词块而非读取磁盘 . . . 15
6.14 真实源码定位：记忆必须服从业务事实 . . . 16
6.15 练习与参考：扩展之前先验证边界 . . . 17
6.16 面试表达：说清一条边界，比堆词更有效 . . . 18
6.17 源码图解：从函数签名读到返回值 . . . 19
6.18 真实接口图：把后端符号与请求对上 . . . 20

每个实验包含全部导入、输入与执行。固定模型和教学数据只验证机制；源码节选只用于阅读。请先预测输出，再运行并改变一个输入。

```
Set-Location D:\Study\LR-Agent
$env:PYTHONPATH = Join-Path (Get-Location) 'LR-Agent\vendor\local-agent'
$env:LANGSMITH_TRACING = 'false'
$env:LANGCHAIN_TRACING_V2 = 'false'
$agentPy = 'D:\Study\LR-Agent\.venvs\langchain-main\Scripts\python.exe'
```

6.1 先问缺什么能力，再决定如何扩展

场景：主 Agent 已能完成工具循环，现在用户需要查工作区规范、委派大范围代码阅读，并在多轮任务中保留偏好。三件事看起来都叫“扩展”，实际职责不同：MCP 统一工具连接，子代理隔离一段查阅任务，上下文和记忆管理模型这次能看到的信息。

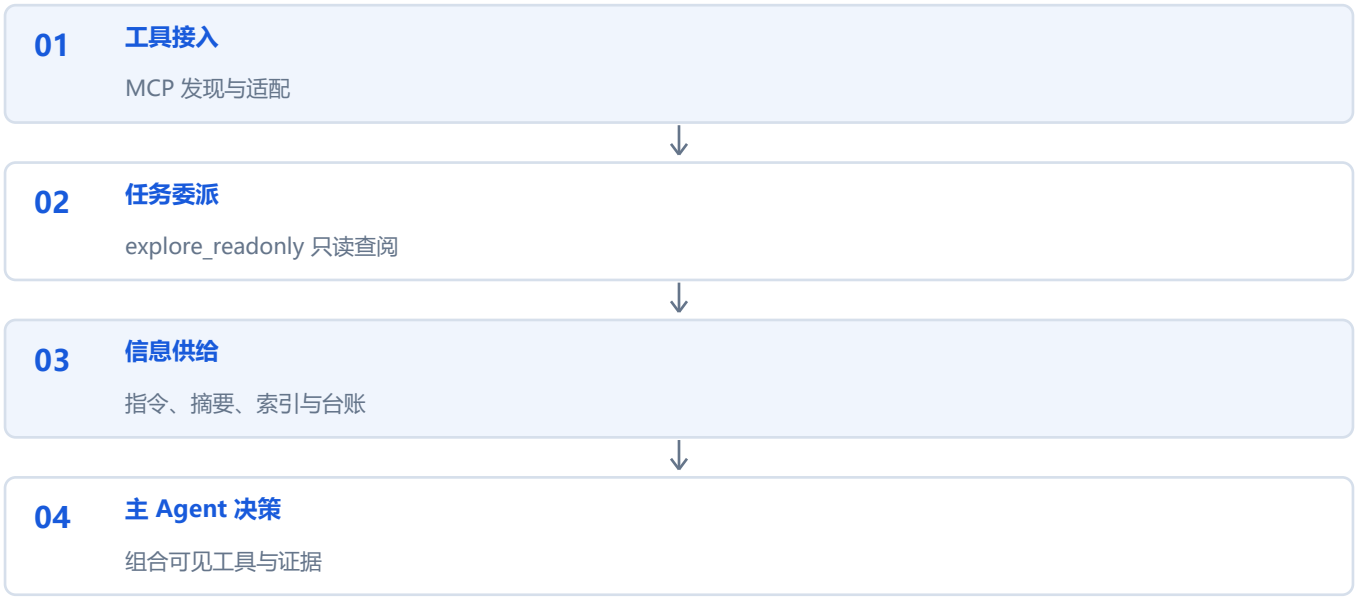


图6-1 概念地图：三类扩展分别解决什么

需求	优先看哪里	本章不夸大的能力
接入已有工具服务	mcp_client.py	不等于自动获得正确权限
减少主循环阅读负担	explore_readonly.py	不是完整多 Agent 团队
记住规范与当前进度	context_snapshot.py	不是向量知识库或图 Store

本项目仍以单个主 Agent 加只读委派工具为主。不因为课程讲过 RAG 就增加检索流水线，也不因为有多个模型调用就宣称 Planner、Executor、Reviewer 架构。本章先学可检查的工程边界，再讨论性能和成本。

6.2 MCP：协议、服务和工具不能混为一谈

MCP 即 Model Context Protocol，模型上下文协议。可以把它理解为工具服务之间的约定：客户端先连接服务器、发现工具描述，再按协议调用工具。MCP Server 是提供工具的一方，MCP Client 是连接它的一方；具体业务工具才真正执行读文件、读记忆等动作。

名词	白话解释	本项目角色
MCP 协议	双方共同说的格式	约定发现与调用方式
MCP Server	提供工具的服务	本机或远程工具源
MCP Client	连接并查询工具的一方	MultiServerMCPClient
Adapter	把一种接口换成另一种	StructuredTool 适配

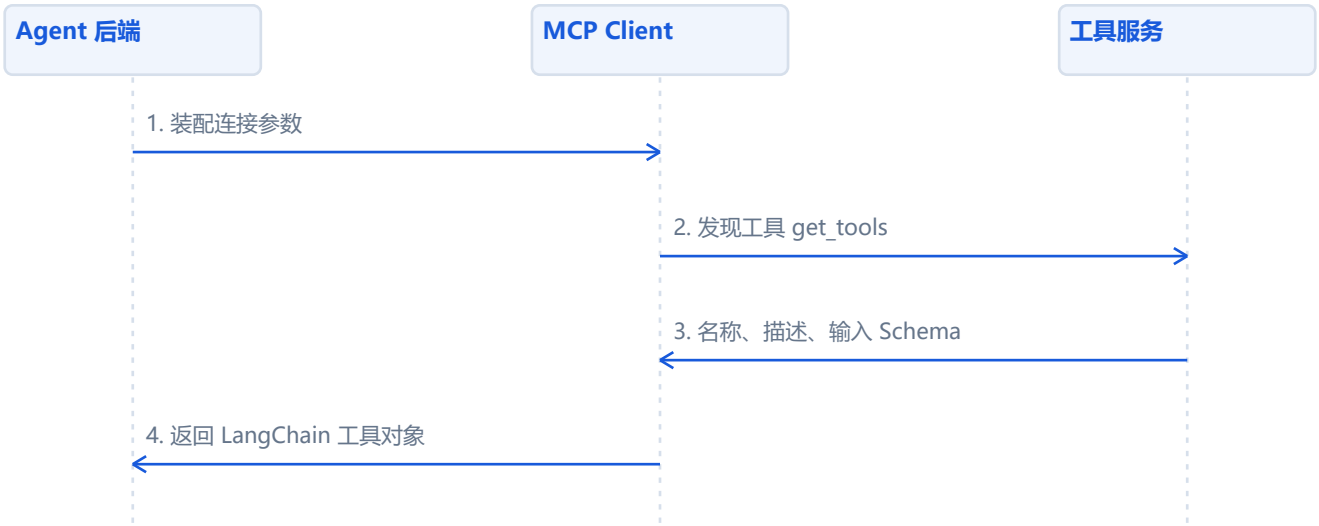


图6-2 实际工具发现路径的简化时序

langchain-mcp-adapters 把发现结果变成 LangChain 可用的工具。发现只拿到“有哪些工具和怎样调用”，不表示每个工具都已经执行过。工具描述还可能很长或存在冲突，因此项目在注入前继续做过滤和去重。

6.3 从配置到工具清单：四步读懂装配

生产入口 merge_mcp_tools 接收请求、Settings 和内置工具列表，返回新的工具列表，不原地修改传入列表。它根据 client_context 中的本机或远程连接配置去发现工具，然后过滤记忆权限、排除与已有名称重复的工具，最后按名称排序。



图6-3 实际装配流程的教学简化

配置	服务端怎样解释	常见误区
本机 URL	末尾补 /mcp	不能照搬远程拼接规则
远程 URL	按配置原样使用	不自动补相同端点
disabled_tools	先排除指定名称	缓存命中也会重新过滤
transport	streamable_http 或 sse	与业务 SSE 事件不是同一概念

本机或远程的传输负责 MCP 通信；第 5 章的 text_delta/tool_pending 是后端给调用方的业务事件，两条链路用途不同。连接配置可能包含凭据，教材只讲字段职责；真实截图和日志都不展示 headers、token 或模型密钥。

6.4 缓存与降级：别让扩展拖住每次请求

每轮都向每台服务重新发现工具，会增加首段输出等待时间。项目用 TTL 缓存保存原始工具列表；TTL 是 Time To Live，缓存有效时长。连接指纹包含 URL、传输方式和排序后的请求头；禁用项过滤与跨服务去重发生在取缓存之后，因此修改禁用项不必等缓存过期。



图6-4 实际发现缓存分支

情况	本轮行为	权衡
首次无缓存	并行发现各服务器	仍需等待网络返回
旧缓存过期	先用旧列表继续	工具清单可能短暂过时
单台发现失败	跳过该服务	可用扩展能力减少
后台刷新失败	保留旧列表	需要日志与后续恢复

这种策略称为 stale-while-revalidate：旧值先服务，后台再验证。它优化工具发现的等待，不是缓存模型答案。源码存在这个机制，不代表已经量化了延迟提升；面试若写性能数据，应记录相同配置下的冷启动与热缓存分布。

6.5 去重与能力：当前保证到哪一步

两个服务器都叫 read_agent_skill，该保留谁？当前发现流程按连接顺序处理，seen_names 记录已收录名称，同名后到者跳过；合并内置工具时也会排除已有名称。能力去重则是预留的一层设计：如果能将工具映射成同一种业务能力，便可避免不同名称重复暴露。

检查	当前实现	边界
服务器禁用项	按 disabled_tools 排除	只匹配指定名称
跨服务同名	seen_names 去重	先收录的名称优先
内置与 MCP 同名	合并前检查已有集合	不会保留两个同名工具
语义能力冲突	有接口，推断当前为 None	不能声称能识别所有同义工具

源码节选：LR-Agent/vendor/local-agent/app/agent/tools/mcp_client.py::_infer_mcp_capability（省略文档字符串）

```
def _infer_mcp_capability(
    tool_name: str,
) -> ToolCapability | None:
    _ = tool_name
    return None
```

这里的 _ = tool_name 表示当前保留该形参但尚未使用；返回 None 表示没有推断出的能力。不能因为存在 ToolCapability 类就把“语义级自动冲突检测”写进简历。可改造任务是登记一个真实重叠能力，并用冲突输入验证修改前后行为。

6.6 完整实验一：记忆开关改变哪些工具

完整教学实验 ch06_memory_policy。在 study 自行新建 my_ch06_memory_policy.py，复制全部代码。

```python
from app.agent.tools.mcp_client import (
    should_expose_mcp_tool,
)
names = ['memory_read', 'memory_write', 'read_agent_skill']
def exposed(enabled, mode):
    return [name for name in names
            if should_expose_mcp_tool(
                name, workspace_memory_enabled=enabled,
                agent_mode=mode,
            )]
disabled = exposed(False, 'annotation')
readonly = exposed(True, 'chat')
editing = exposed(True, 'annotation')
assert disabled == ['read_agent_skill']
assert readonly == ['memory_read', 'read_agent_skill']
assert editing == names
print('off:', disabled)
print('ask:', readonly)
print('agent:', editing)
```

运行：& $agentPy ".\study\my_ch06_memory_policy.py"（工作区根目录）

本轮实际输出：

```
off: ['read_agent_skill']
ask: ['memory_read', 'read_agent_skill']
agent: ['memory_read', 'memory_write', 'read_agent_skill']
```

exposed 是教学封装；enabled 和 mode 分别控制工作区记忆开关与交互模式。列表推导式对每个 name 调用项目真实策略函数，只有返回 True 的名称被保留。

开关关闭时 memory_* 都隐藏；开启但不在 annotation 模式时只暴露 memory_read。read_agent_skill 不属于 memory_*，因此不受这个记忆开关过滤。

该实验只验证名称暴露策略，不连接 MCP、不读取真实记忆。工具可见不等于有权在执行端写任意路径；具体服务仍需执行侧约束。

6.7 只读子代理：把一次查阅封装成工具

子代理是被主 Agent 委派、为一段子任务运行模型和工具的执行单元。本项目 explore_readonly 适合“先摸清目录里哪些代码与任务相关”，结果返回中文摘要。主模型把它当作一个普通 StructuredTool 调用；工具体内运行 ExploreReadonlyRunner 的短循环。

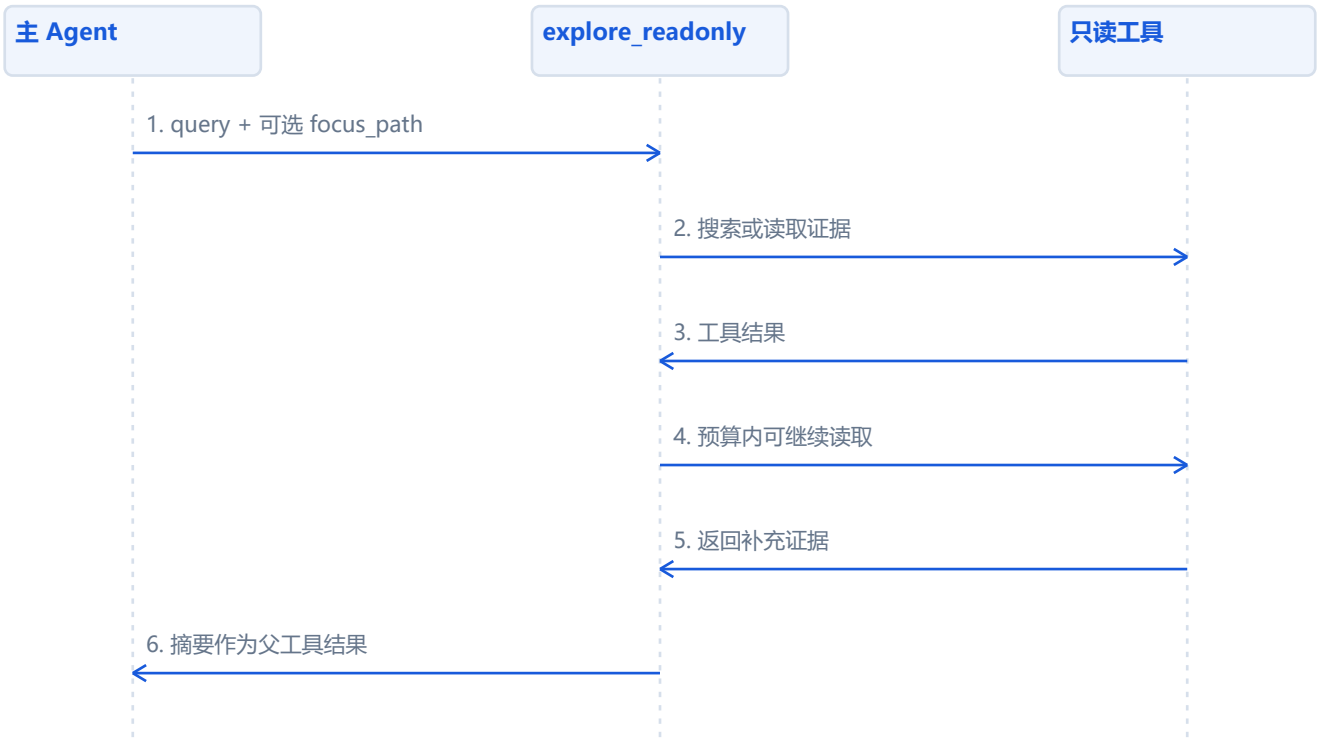


图6-5 实际只读委派的简化时序

输入或结果	解释	使用方式
query	要查的问题，必填	给清楚目标与判定标准
focus_path	可选的查阅重点软提示	优先关注；非硬路径限制
summary	子任务结论	父 Agent 判断后继续任务
subagent_* 事件	可观察的内部进展	不全量塞入父消息链

它不负责改文件、写标注或做任意委派。真正的写任务仍由主 Agent 调用对应工具并经过业务门控。把输入写成“查出标注工具的调用路径并列出文件依据”比“帮我看看项目”更容易得到可验证摘要。

focus_path不是访问边界

当前实现把focus_path拼成“优先关注路径”提示，不强制子代理只读该路径。真正边界由工具白名单和底层工作区路径校验决定。

6.8 只读限制：工具清单与执行表一起过滤

只在系统提示里写“禁止写文件”不够。模型可能提出错误调用，所以只读子代理使用白名单装配：工具描述列表和实际函数映射都被过滤。只读白名单包括搜索、列目录、读文件和帮助；视当前模式，可增加已有标注读取。递归调用 explore_readonly 和写入类工具都不可进入。



图6-6 实际双过滤机制

候选工具	只读查阅能否获得	原因
read_workspace_file	可以	只读白名单
read_file_annotation	按上下文决定	标注读取可选
write_workspace_file	不可以	写文件
auto_annotate / memory_write	不可以	写标注或记忆
explore_readonly	不可以	避免递归委派

白名单是允许哪些函数，不是操作系统级沙箱。底层读工具仍需约束路径、编码和结果长度。子代理还有独立工具轮次预算与取消检查；单工具结果截断有助于限制上下文膨胀，但也可能遗漏证据，因此摘要应保留可核对路径。

6.9 完整实验二：亲眼看双白名单生效

完整教学实验 ch06_readonly。在 study 自行新建 my_ch06_readonly.py，复制全部代码。

```python
from langchain_core.tools import StructuredTool
from app.agent.assist.explore_readonly import (
    filter_explore_inner_tools, filter_explore_inner_fn_map,
)
def demo():
    return 'not executed'
names = ['read_workspace_file', 'write_workspace_file',
         'auto_annotate', 'read_file_annotation']
tools = [StructuredTool.from_function(
    func=demo, name=name, description='offline demo',
) for name in names]
selected = filter_explore_inner_tools(
    tools, include_annotation_reads=False,
)
functions = filter_explore_inner_fn_map(
    {name: demo for name in names},
    include_annotation_reads=False,
)
assert [x.name for x in selected] == ['read_workspace_file']
assert list(functions) == ['read_workspace_file']
print('tools:', [x.name for x in selected])
print('functions:', list(functions))
```

运行：& $agentPy ".\study\my_ch06_readonly.py"（工作区根目录）

本轮实际输出：

```
tools: ['read_workspace_file']
functions: ['read_workspace_file']
```

StructuredTool.from_function 把普通函数包装为带名称和 Schema 的工具。这里所有函数都返回固定文字，且没有调用它们；实验只考察装配阶段的过滤。

filter_explore_inner_tools 控制子代理看见的工具；filter_explore_inner_fn_map 控制实际可分派的函数。两边同时过滤，避免只隐藏说明却仍可执行写操作。

include_annotation_reads=False 还会排除 read_file_annotation。改成 True 可观察允许集合扩大，但写文件和 auto_annotate 仍不可进入。实验不调用模型，不证明模型一定生成高质量摘要。

6.10 custom 通道：进度流与父消息分开走

如果把子代理每条 token 都当作父回答输出，用户会看到来源混杂的文本。如果把内部读取全文全加到父消息，又会放大上下文。当前包装层让子代理进展通过 runtime.stream_writer 写入 custom 通道，最终只有摘要变成父工具结果。

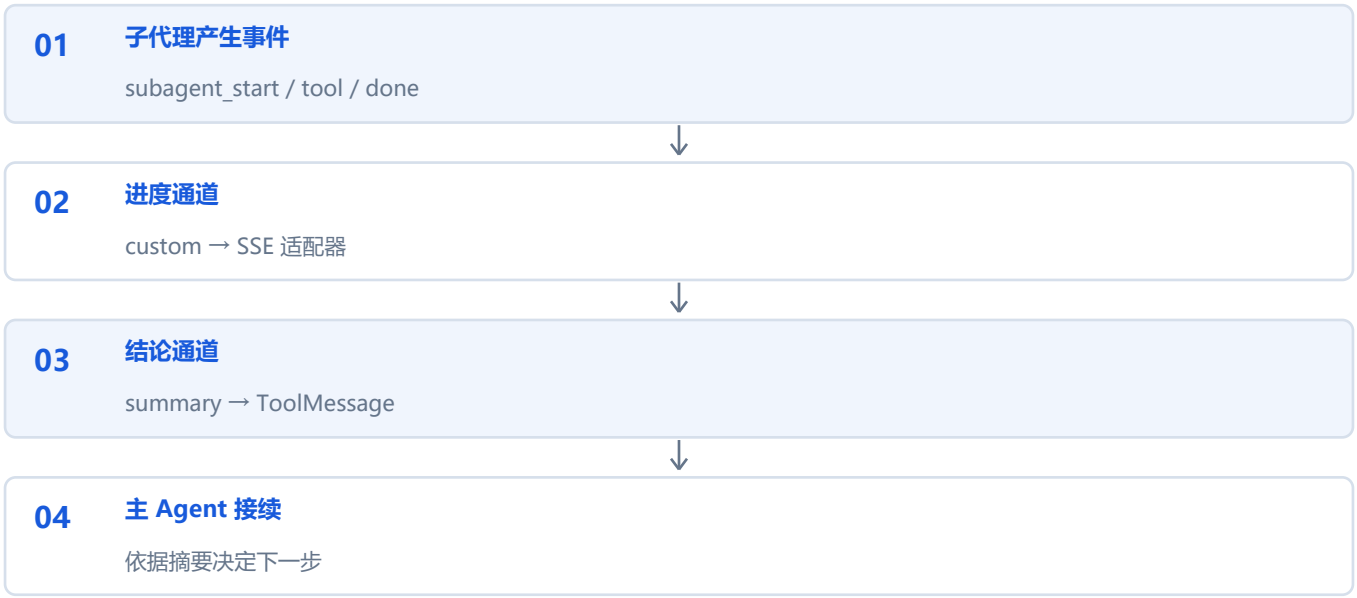


图6-7 实际双通道：轨迹与摘要分离

通道或字段	作用	排错重点
messages	模型消息与增量	避免子 token 污染父回答
custom	自定义子代理事件	装配必须显式开启
tool_call_id	父委派调用 ID	与外层工具对齐
inner_tool_call_id	内部读取调用 ID	不能与父 ID 混用

当前装配包含 messages、updates、custom。工具拿到可调用 writer，也不能单独证明外层已经订阅 custom；外层漏配时可能静默过滤帧。因此项目在装配处校验，而非仅在工具函数里检查 writer。这里讲的是本项目适配选择，不是宣称所有 LangChain 场景都禁止嵌套 create_agent。

6.11 上下文、摘要、记忆、Store 逐个分清

Context 是本次调用带入的运行信息，例如工作区与已完成 ID；State 是图运行中会变化的消息和回合字段。摘要是把较早对话压成文字，减少这次给模型的历史量；记忆则是被业务保存、供后续任务读取的规范或偏好。它们可以配合，但不是同一对象。

概念	LR-Agent 中的位置	不能等同
AgentContext	单次调用注入	长期数据库
BusinessRoundState	图内消息与回合字段	跨请求持久恢复
context_summary	请求携带的此前摘要	后端自动恢复的完整历史
Markdown 记忆 / SQLite	业务文件与会话存储	LangGraph Store
图 Checkpointer	当前工厂未传入	业务撤销快照

后端 _build_lc_messages 把 context_summary 加为一条标有“此前对话摘要”的 SystemMessage，再转换这次请求携带的消息。该函数消费摘要，不负责在后台自动替你压缩或持久化图。摘要可能丢失细节，重要调用 ID、未确认状态和工具证据应有明确的结构化保留策略。

常见面试陷阱

“用了 SQLite，所以 Agent 有持久记忆”过于含糊。应说明保存的是什么、谁写入、何时读取、如何进入本次模型输入，以及是否真正接入了图 checkpointer / Store。

6.12 指令、Skills 和记忆索引如何进提示词

模型不会天然知道项目目录里的规范。build_assist_system_prompt 按输入快照拼装系统提示：身份与任务规则在前，再放项目指令、Skills 目录、启用的记忆索引，最后是动态台账。稳定信息靠前、动态信息靠后，有利于维持长前缀，但缓存收益需单独测量。



图6-8 实际提示词信息层次的简化视图

块	携带什么	后续如何拿细节
项目指令	项目级行为要求	直接进入系统提示
Skills 目录	名称 + 描述	read_agent_skill 读取正文
记忆索引	topic 入口与摘要	memory_read 按需查阅
提案台账	待确认或已处理事实	结合结构化状态与工具结果

Skills 目录是 catalog，不是把每个 SKILL.md 全文塞入模型。提示词要求先读正文，再按需读附属文件；脚本只可阅读，不可执行。记忆索引也只是入口，不代表模型已经读过所有 topic，更不构成向量检索流水线。

6.13 完整实验三：验证提示词块而非读取磁盘

完整教学实验 ch06_prompt_blocks。在 study 自行新建 my_ch06_prompt_blocks.py，复制全部代码。

```python
from app.agent.context_snapshot import build_assist_system_prompt
from app.schemas.agent import (
    ClientContextInput, SkillCatalogEntryInput,
)
ctx = ClientContextInput(
    project_instructions='演示规范：先说明依据',
    skills_catalog=[SkillCatalogEntryInput(
        name='demo_skill', description='读取标注规范',
    )],
    workspace_memory_enabled=True,
    memory_index='演示索引：preferences.md',
)
prompt = build_assist_system_prompt(ctx, model='offline-demo')
titles = ['【项目指令】', '【可用 Skills】', '【工作区记忆】']
positions = [prompt.index(title) for title in titles]
assert positions == sorted(positions)
assert 'demo_skill：读取标注规范' in prompt
assert '演示索引：preferences.md' in prompt
assert '先调用 read_agent_skill' in prompt
print('order:', ' -> '.join(titles))
print('catalog and index: present')
print('skill execution: not performed')
```

运行：& $agentPy ".\study\my_ch06_prompt_blocks.py"（工作区根目录）

本轮实际输出：

```
order: 【项目指令】 -> 【可用 Skills】 -> 【工作区记忆】
catalog and index: present
skill execution: not performed
```

ClientContextInput 是输入快照，不会因为填了 memory_index 就读取磁盘。SkillCatalogEntryInput 只有名称和描述；完整 Skill 内容需要后续工具按需读取。

str.index 返回子串第一次出现的位置；用递增位置检验块顺序。build_assist_system_prompt 返回提示词字符串，不会调用模型，也不会执行 Skill 脚本。

本实验用“演示”内容验证目录与索引注入。稳定块靠前有利于长前缀复用，但该实验没有测 token 消耗、服务商缓存命中或长会话任务成功率。

6.14 真实源码定位：记忆必须服从业务事实

项目记忆用于规范、偏好和可回读的任务事实。progress.md、annotated-files.md 由系统在确认或标注落盘后更新；Agent 的 memory_create / memory_write 主要用于用户规范、偏好与纠正，不能把未确认提案记成已完成，也不能凭印象改计数。

源码节选：LR-Agent/vendor/local-agent/app/agent/tools/mcp_client.py::should_expose_mcp_tool（函数体节选）

```python
if tool_name.startswith("memory_"):
    if not workspace_memory_enabled:
        return False
    if agent_mode != "annotation":
        return tool_name == "memory_read"
    return True
return True
```

要解释的行为	源码入口	读完回答
记忆工具暴露	mcp_client.py::should_expose_mcp_tool	关闭、Ask、Agent 有何区别?
只读子代理权限	explore_readonly.py::filter_explore_inner_fn_map	执行侧为什么也过滤?
目录与索引注入	context_snapshot.py::build_assist_system_prompt	哪些内容只是入口?
自定义事件桥接	subagent.py::build_explore_readonly_agent_tool	轨迹与摘要如何分流?

建议真实截图：mcp_client 的暴露策略与 explore_readonly 白名单；只显示代码，不打开用户记忆文件。SQLite 会话记录、Markdown 记忆和业务撤销快照都是真实业务存储，但它们不会因为存在于项目里，就自动成为 LangGraph Store 或 checkpointer。

6.15 练习与参考：扩展之前先验证边界

练习	判断	参考答案
关记忆后 Skill 还能用吗？	能否一刀切全部隐藏?	Skill 不属于 memory_*，策略不同
给子代理加一个写工具	只改提示词够吗?	不够，保持工具与函数双过滤
两台 MCP 同名工具	是否同时注入?	当前后到者跳过，顺序需明确
摘要写“已保存”但待确认	信文字还是结构化状态?	核对状态与实际工具事实

动手一：实验二将 include_annotation_reads 改成 True，分别观察工具与函数表，确保只多出允许的标注读取。动手二：实验三将 workspace_memory_enabled 改为 False，应不再出现工作区记忆块，但 Skills 目录仍在。修改断言时应说明验证目标，不能只删掉失败断言。

设计讨论：如果扩展服务临时断连，应让主任务整体失败，还是隐藏该服务继续？参考：项目发现阶段倾向单台失败降级。核心任务不可缺的工具应给清楚提示；非核心扩展可以继续，但不能让模型假装工具仍可用。结果依赖特定工具时，需要单独的能力可用性判断和观测。

可留下的个人证据

记录自己运行的暴露矩阵、双白名单对比和提示词块顺序。若亲自改造，再补冲突输入、失败用例、本人提交与回归结果；不要把已有 MCP 接入都写成本人从零开发。

6.16 面试表达：说清一条边界，比堆词更有效

30 秒概念回答：MCP 解决工具发现与接入，子代理解决受限的子任务委派，记忆解决信息保留和再次注入。LR-Agent 用 MCP 适配动态工具，用只读 explore_readonly 查阅工作区，目录与索引先进入提示词、细节按需读取。它没有产品 RAG，也没有因此自动获得图持久状态。

2 分钟项目回答框架：从“主 Agent 要查规范、但不该把所有文档塞入上下文”开始；讲 MCP 发现、缓存、去重和记忆暴露策略，再讲子代理工具/执行表双白名单、预算和摘要返回；最后区分上下文摘要、业务存储和 LangGraph Store。配一张真实源码截图和两个自己的实验输出即可形成可信证据。

追问	回答核心	尚需验证
为什么先用单主 Agent?	结构更直接，委派局部阅读	复杂任务是否需更多角色
缓存过期为何用旧列表?	降低发现阻塞，接受短暂陈旧	真实延迟与失效风险
只读是否等于绝对安全?	双白名单限制工具，仍需路径约束	底层服务权限和失败输入
能否写提升准确率或降成本?	先做对照实验再量化	样本、模型、成本和质量

本章事实边界

源码已确认：发现缓存、名称去重、暴露策略、只读委派和提示词块。本次已执行：三个离线函数实验。未连接真实 MCP、未执行子代理模型、未读写真实记忆；不据此宣称延迟、成本或摘要质量提升。

6.17 源码图解：从函数签名读到返回值

先看图中的真实路径与原始行号，再打开源文件跟读。签名说明输入条件，函数体说明处理，return说明交给下一层的数据。图为源码阅读辅助，不替代完整上下文。

真实源码阅读图 / resolve_explore_inner_names

LR-Agent/vendor/local-agent/app/agent/assist/explore_readonly.py

```
68  def resolve_explore_inner_names(*, include_annotation_reads: bool) -> frozenset[str]:
69      names = set(EXPLORE_READONLY_BASE_INNER)
70      if include_annotation_reads:
71          names |= EXPLORE_READONLY_ANNOTATION_INNER
72      return frozenset(names - EXPLORE_READONLY_FORBIDDEN_INNER)
```

真实源码阅读图：LR-Agent/vendor/local-agent/app/agent/assist/explore_readonly.py::resolve_explore_inner_names。按当前文件原始行号截取并排版生成。

先看什么	怎样读	自己记录什么
输入	区分实参、配置与上下文	哪些字段可缺失
处理	找条件、调用和错误分支	哪一步改变消息或执行
输出	确认返回值与接收者	怎样用实验检查

6.18 真实接口图：把后端符号与请求对上

本轮从当前源码启动了仅监听本机的文档实例，截图来自真实Swagger页面。此处只读查看，没有提交模型请求、推理任务或个人凭据。动态端口不属于协议语义。



本轮真实截图：当前源码启动的Local Agent Swagger接口总览，只读查看，未提交模型或标注请求。

图中信息	对应后端问题	继续核对
接口路径	由哪个函数接收请求	查路由装配与前缀
请求结构	模型能读到哪些数据	查Schema与消息转换
状态与结果	什么是完成、失败或挂起	查SSE与续跑合同

源码与证据索引

LR-Agent/vendor/local-agent/app/agent/tools/mcp_client.py::load_mcp_tools_from_servers/should_expose_mcp_tool/_infer_mcp_capability

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py::merge_mcp_tools/_build_lc_messages

LR-Agent/vendor/local-agent/app/agent/assist/explore_readonly.py::ExploreReadonlyRunner/filter_explore_inner_tools/filter_explore_inner_fn_map

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/subagent.py::build_explore_readonly_agent_tool/subagent_event_payload

LR-Agent/vendor/local-agent/app/agent/context_snapshot.py::format_skills_catalog_block/build_assist_system_prompt

LR-Agent/vendor/local-agent/app/schemas/agent.py::ClientContextInput/SkillCatalogEntryInput

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/runtime.py::AgentContext/build_langchain_agent

阅读时先定位符号，再看完整输入、处理与返回。源码存在、离线实验通过和已有真实验收属于不同证据；未验证的模型质量、效率与并发结论仍需单独实验。